

Perlの幻影に育てられてきた 俺たちは

Raised by the Phantom of Perl



pixiv Inc.
USAMI Kenta

pixiv

お前誰よ



- うさみけんた (@tadsan) / Zonu.EXE / にゃんだーすわん
- ピクシブ株式会社 Platform Div > WebTechnology Team PHPer
 - 2012年末から現職、APIとかCIとかいろいろなところを見つめてきました
 - 最近チームが再編されてインフラっぽい仕事もしてます
- Emacs PHP Modeを開発しています (2017年-)
- プログラミング言語にちょっとこだわりのある素人 (spcamp2010)

PHPとか
Emacs Lispを
やらせていただきます



にゃんだーすわん
クリエイターページ

投稿する

ダッシュボード

投稿管理

プラン管理

クリエイター設定

ファン一覧

おたより

支援金管理/振込

FANBOXプリント

FANBOXプリントとは

コンテンツ提供者申請

クリエイター優待

Rigor Playground No errors

```
1  # Rigor Playground – edit this code and see diagnostics in real time
2
3  def greet(name)      String
4    "Hello, " + name   String
5  end                  :greet
6
7  world = if rand == 0  false | true
8            "World"     "World"
9            else
10             "Ruby"     "Ruby"
11          end          "Ruby" | "World"
12
13  result = greet(world) "Hello, Ruby" | "Hello, World"
14  result.upcase         "HELLO, RUBY" | "HELLO, WORLD"
15
```

ちょっとすごいRubyの型チェッカーを作ります 編集

2026年5月25日 23:20 · 全体公開

これは自画自賛しても許されるくらいちゃんとすごい技術だと思っているのですが...

私はLispやRubyのような動的言語が大好きで、2016年くらいからPHPに型をつけながら暮らしている者です。もともとは2010～2014年くらい前までShibuya.rbやRuby会議に顔を出していたものの、仕事でPHPをやっていることもあってPHP方面に軸足を移していました。

YAPC::Asia Tokyo
2012-2015
YAPC::Hokkaido
2016 SAPPORO
builderscon 2017-





yapcJapan
@yapcJapan



[Show translation](#)

札幌地域の降雪により、会場となる札幌産業振興センター近辺はかなり積雪しております。ご来場の際はお足元にご注意の上、お怪我などされないようお気をつけください [#yapcJapan](#)



9:03 AM · Dec 10, 2016



2016 mid in Shinagawa

"P" is not "Perl", "P" is "Pachimon".

[写真や録画 / 感想エントリリンク集](#)

[Togetter: 開催前 / 7/2\(土\) / 7/3\(日\) / 開催後](#)

[トップ / 目標 / トーク / 飛び入りトラック / 参加 / スタッフ / 主催 / Feed](#)

[タイムテーブル 7/2\(土\) / 7/3\(日\)](#)

[会場の情報、諸注意 / 登壇者向け情報](#)

[twitter:@yapcasia8oji](#) [Hashtag:#yapc8oji](#)



わたしとパール

時は2003年くらい

HTML/Web標準に 興味がある学生

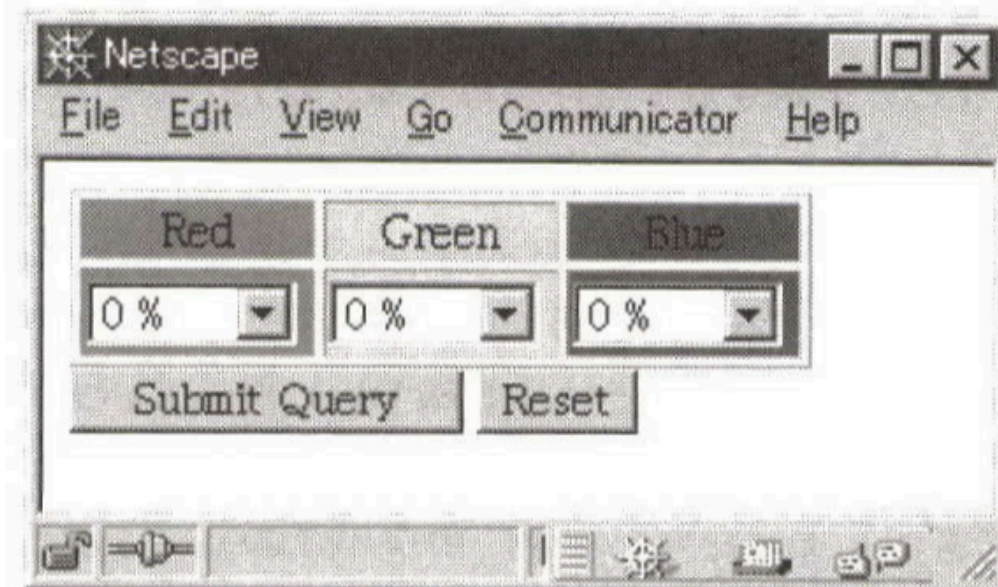
人生で初めて買った技術書



CGIをはじめよう

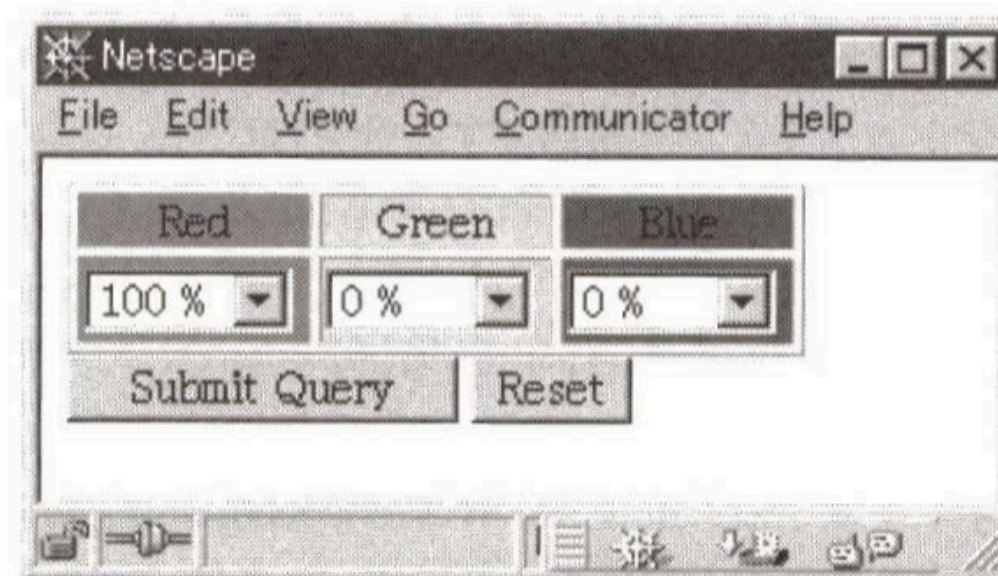
ここでは、簡単なCGIアプリケーションを例に取って、基本的なCGIのプログラミング方法の説明を行います。サンプルとするCGIアプリケーションは、ブラウザでRGBのパーセンテージで各色を指定して、16進数でその指定した色を表示すると同時に背景色として指定色を表示するものです。まず、その動きを見てみましょう。ブラウザで**renshu.html**というHTMLファイルを指定すると、次のように10%刻みで、Red、Green、Blueの3色を指定できる画面が表示されます。

▼図1 初期入力画面



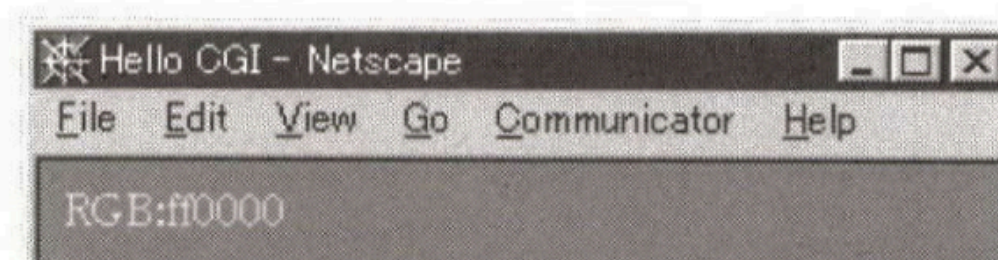
Redを100%にセットしてみます。

▼図2 色指定後の画面



指定後、**Submit Query**ボタンを押してWWWサーバに送信します。送信データはCGIプログラム**getCol.cgi**で処理され、その出力がWWWサーバから返されます。受け取った画面は次のようになります。

▼図3 CGI処理後の表示画面



最初に表示を行うHTMLファイル**renshu.html**は、以下のようになっています。

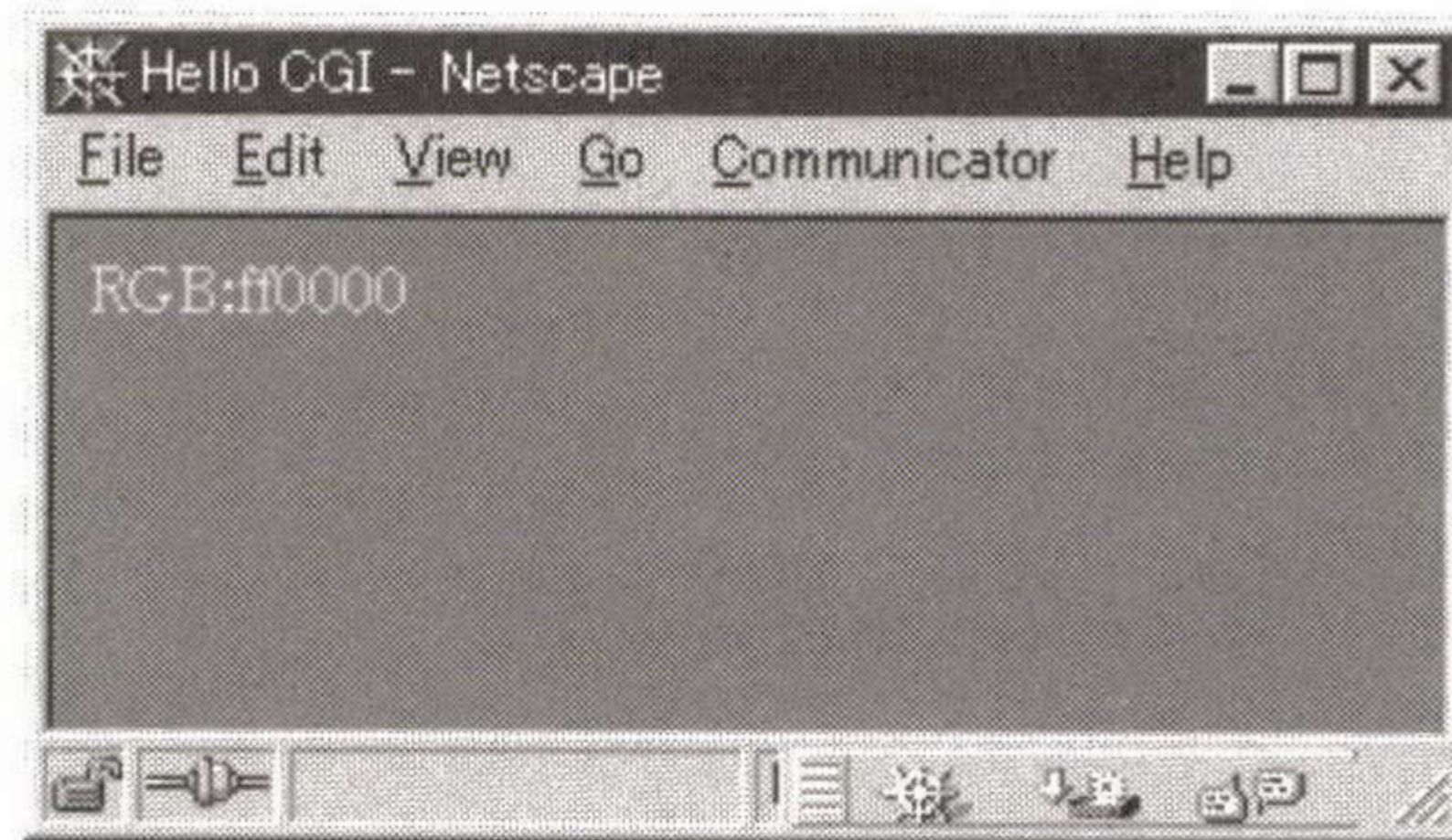
▼renshu.html

```
<HTML>
<HEAD></HEAD>
<BODY>
<FORM NAME="colorGenelator" ACTION="/cgi-bin/genCol.cgi">
<TABLE BORDER="1">
<TR ALIGN="center">
<TD BGCOLOR="#ff0000">Red</TD>
<TD BGCOLOR="#00ff00">Green</TD>
<TD BGCOLOR="#0000ff">Blue</TD>
</TR>
<TR>
<TD BGCOLOR="#ff0000">
<SELECT NAME="Red" SIZE="1">
<OPTION VALUE="0" selected>0 %</OPTION>
<OPTION VALUE="10">10 %</OPTION>
<OPTION VALUE="20">20 %</OPTION>
<OPTION VALUE="30">30 %</OPTION>
<OPTION VALUE="40">40 %</OPTION>
<OPTION VALUE="50">50 %</OPTION>
<OPTION VALUE="60">60 %</OPTION>
<OPTION VALUE="70">70 %</OPTION>
<OPTION VALUE="80">80 %</OPTION>
<OPTION VALUE="90">90 %</OPTION>
<OPTION VALUE="100">100 %</OPTION>
</SELECT>
</TD>
<TD BGCOLOR="#00ff00">
<SELECT NAME="Green" SIZE="1">
<OPTION VALUE="0" selected>0 %</OPTION>
<OPTION VALUE="10">10 %</OPTION>
<OPTION VALUE="20">20 %</OPTION>
<OPTION VALUE="30">30 %</OPTION>
<OPTION VALUE="40">40 %</OPTION>
<OPTION VALUE="50">50 %</OPTION>
<OPTION VALUE="60">60 %</OPTION>
<OPTION VALUE="70">70 %</OPTION>
<OPTION VALUE="80">80 %</OPTION>
<OPTION VALUE="90">90 %</OPTION>
<OPTION VALUE="100">100 %</OPTION>
</SELECT>
</TD>
<TD BGCOLOR="#0000ff">
<SELECT NAME="Blue" SIZE="1">
<OPTION VALUE="0" selected>0 %</OPTION>
<OPTION VALUE="10">10 %</OPTION>
<OPTION VALUE="20">20 %</OPTION>
<OPTION VALUE="30">30 %</OPTION>
<OPTION VALUE="40">40 %</OPTION>
<OPTION VALUE="50">50 %</OPTION>
<OPTION VALUE="60">60 %</OPTION>
<OPTION VALUE="70">70 %</OPTION>
<OPTION VALUE="80">80 %</OPTION>
<OPTION VALUE="90">90 %</OPTION>
<OPTION VALUE="100">100 %</OPTION>
</SELECT>
</TD>
</TR>
</TABLE>
<INPUT TYPE="submit">
```

これはCSSの本だと誤解してもおかしくないのでは…？

指定後、**Submit Query**ボタンを押してWWWサーバに送信します。送信データはCGIプログラム**getCol.cgi**で処理され、その出力がWWWサーバから返されます。受け取った画面は次のようになります。

▼図3 CGI処理後の表示画面



▼genCol.cgi

```
#!/usr/bin/perl
*data = &parseInput($ENV{'REQUEST_METHOD'}); .....①
$red = 255 * ($data{'Red'}/100); .....②
$green = 255 * ($data{'Green'}/100);
$blue = 255 * ($data{'Blue'}/100);
$bgcolor = sprintf("#%02x%02x%02x", $red, $green, $blue);
$fgcolor = sprintf("#%02x%02x%02x", $red^255, $green^255, $blue^255);

print "Content-type: text/html\n\n";
print "<HTML>\n";
print "<HEAD><TITLE>Hello CGI</TITLE></HEAD>\n";
print "<BODY BGCOLOR=\"\$bgcolor\">\n";
print "<FONT COLOR=\"\$fgcolor\">\n";
printf "RGB:%02x%02x%02x\n", $red, $green, $blue;
print "</FONT>\n";
```

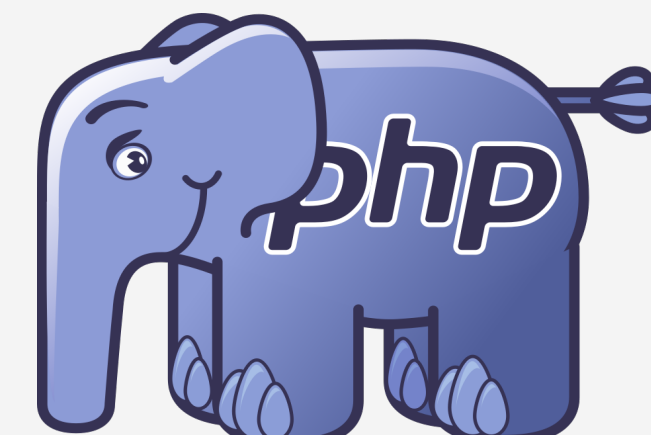
ど素人すぎる

よくわからんが
KENT WEBは便利

SNSがない時代
俺たちはレンタルサーバに
掲示板を設置して
交流を図っていた

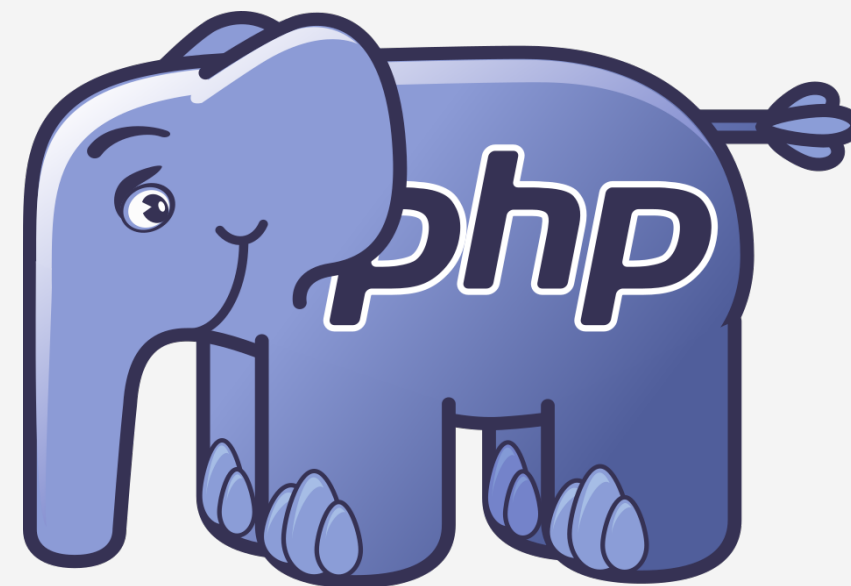
たぶん現代で
それが可能なのは
ほぼPHP

Perlに強い影響を
受けて設計された
言語を使い続けてきた



Perlの間接的な ユーザーの立場で 見守ってきた





それぞれの言語が
Perlから継いだ要素が
三者三様すぎる

なぜPerlが
重宝されたのか

Perlの #ラーコンテンツ

CGI

Common Gateway Interface

- HTTPサーバーがリクエストを受ける度にプロセスを起動する
 - リクエストヘッダが環境変数に、リクエストボディがstdinに入力される
 - プログラムがstdoutに出力したものがヘッダ・ボディになる
- 標準入出力と環境変数さえ読み書きできれば言語はなんでもいい
 - 現代でもAWKとかBashとかでCGIを書く人がいたりする

C言語で
Web開発したくない

Webは本質的に
入出力テキスト処理

単純な好みだけでなく
Cでメモリ安全な
文字列処理は難しい

Perlはもともと
テキスト処理に
向いていた

リクエストごとに
プロセスを起動するので
パフォーマンスの問題
(時間/メモリ/C10K)

CGIスクリプティング
言語としてのPerlを
先鋭化させたのがPHP

CGI言語としてのPerl/PHP

- CGI処理に求められているのは本質的にはHTTPヘッダとボディ出力
 - Perl: cgi-lib.pl や CGI.pm などのライブラリでの「出力サポート」
 - 気をつけて使わないとHTTPヘッダが壊れる
 - PHP: HTTP出力そのものが言語コアに統合されている
 - 言語ランタイムがHTTPメッセージの入出力を担保してくれる
 - CGIではない環境でもCGIスタイルのスクリプトが基本

レンタルサーバに
ファイル転送する
だけでデプロイ完了
(CGI Perl, PHP)

Web言語としてのインターフェイス

- CGIはリクエストごとにプロセスをフォークするのでパフォーマンスが出ない
 - リクエストごとに状態がすべてリセット、グローバル変数などは問題がない
- 90年代のアプローチ: Apacheモジュールとして組み込む
 - mod_perl, mod_ruby: デフォルトでは同じプロセスで動き続ける
 - mod_php: リクエストごとに状態が破棄されるCGI互換モードがデフォルト

プロセス内サーバインターフェイス

- CGIのようなstdin/stdoutではなく、同じ言語の関数レベルの仕様
 - 2003年 WSGI (Python) | 2007年 Rack (Ruby)
 - 2009年 PSGI / Plack (Perl) @YAPC::Asia 2009 miyagawa
- CGI世代の「スクリプト」と「フレームワーク／アプリケーション」の分水嶺
 - 分離するとユニットテストが書きやすくなるなどのメリットは大きい
 - PHPにも共通インターフェイスはあるが、必須ではなく断絶を経験しない

言語としてのPHP
表面的に似ているのは
\$ と ; くらい

Perlが 普及させたもの

Hash 正規表現 cpan (リポジトリ)

意外とPerlの
前からある
気がする……？

ハッシュ（連想配列）

- ハッシュテーブルというデータ構造はPerl以前からあった
- 連想配列(つぽいもの)を実現するデータ構造はハッシュ以外にもあった
 - Lisp系は連結リストベースの連想リスト・プロパティリストというものがある
- ライブラリやユーザー定義のデータ構造として流行らせたのはPerlでは…？
- 現代ではJSON(object)を経て、どんな言語でも扱えることが普通

正規表現

- 正規表現はPOSIX基本正規表現、POSIX拡張正規表現がある…が、Perl正規表現はPOSIX系と互換性がない？
- 記号の意味が大きく違うというわけではないが、\ エスケープが反転
- 現在普及している多くの『正規表現』は多かれ少なかれPerl風
- Perlの全機能が網羅されているものはないが、しばしば実用される範囲での互換性はあると思う / PCREはPerl完全互換ではない

CPAN (Comprehensive Perl Archive Network)

- Perlで書いたライブラリを共有する仕組み
- ざっくりRubyGems, NPM, PyPI, crates.ioなどに繋がるもの
- READMEに書くべき内容やテストの標準化など今も尊ばれるべきもの
- Perl界隈でCPANオーサーがリスペクトされる文化が大好き

先行する テキスト処理言語

sed, awk, shellから
Perlが受けた影響

Perlと同世代の 言語との比較

Perl vs オブジェクト指向

Guidoは
Perl 5の
何を嫌ったのか

Perl 6は
結局なんだったのか

どこかで話したい!!
(20分でも明らかに足りない)

採択されませんでした

YAPC::Tokyo 2026

20分

Perl以前と以後の言語学



うさみけんた



tadsan

☆ 4

🗨️ フィードバック

Perlは1987年に登場したプログラミング言語ですが、言語学の背景を持つ作者ラリー・ウォールの思想をもとにプログラミング言語・スクリプト言語の歴史を塗り替えました。

直接的にはPython(1991年)、Ruby(1995年頃)、PHP(1995年)、JavaScript(1995年)などがそれぞれまったく違うものをPerlから受け取って現代まで繋げています。また、CGIによってWeb文化の普及を支えた象徴的な言語でもあります。

このライトニングトーク(20分)では、複数言語とWebフレームワークに触れてきた立場から以下のような内容について語ります。

- Perlが前の世代から受け取ったものと新しく打ち出したもの
- 同世代の言語であるTcl(1988年)との比較
- 後続言語がPerlから何を学びとったのか
- Web・オブジェクト指向とPerl

ご静聴
ありがとうございました